

System Design Interview - Complete Guide

1. Overview

A system design interview is **not about the “correct answer”**.

It evaluates:

- Thinking process
- Trade-off decisions
- Communication skills
- Real-world engineering judgment

2. Core Principles

1. Clarify Requirements (MOST IMPORTANT)

Always start here.

Functional Requirements

- What system should do
- Example: “User can post content”

Non-Functional Requirements

- Scale (QPS)
- Latency (P99)
- Availability
- Consistency

Ask:

- Expected traffic?
- Read vs Write ratio?
- Real-time or eventual consistency?

2. Everything is a Trade-off

There is **no perfect system**

Every decision impacts:

- Performance
- Cost
- Complexity

Example:

- Caching → faster but stale data risk
- Replication → high availability but consistency issues

3. Drive the Interview

Don't wait for interviewer

Instead:

- Suggest next steps
- Guide discussion
- Show structured thinking

4. Time Management (CRITICAL)

You have ~45–60 minutes.

Suggested breakdown:

- 5 min → Requirements
- 10 min → High-level design
- 20 min → Deep dive
- 10 min → Scaling & trade-offs
- 5 min → Wrap-up

5. Always Discuss Observability

Include:

- Logging
- Monitoring
- Alerting
- Auditing

Without this:

Design is incomplete

6. Discuss Maintainability

Include:

- Debuggability
- Code complexity
- Security
- Privacy

7. Failure Handling (HIGH IMPACT)

Always assume failure

Discuss:

- Graceful degradation
- Retry mechanisms
- Circuit breakers
- Timeouts

Golden rule:

“Never trust any component”

8. Use Diagrams

Always draw:

- Architecture diagram
- Data flow
- Sequence diagram

Visual clarity = strong impression

9. System Can Always Improve

Show iterative thinking:

- Start simple
- Scale gradually

3. Ideal Interview Flow

Step 1: Requirements + Trade-offs

- Functional
- Non-functional
- Clarify scope

Step 2: API Design

Example:

None

- `POST /posts`
- `GET /posts/{id}`

Shows practical thinking

Step 3: Data Model

- Tables / Collections
- Relationships

Example:

- Users
- Posts
- Comments

Step 4: High-Level Design

None

Client → Load Balancer → App Servers → DB

↓

Cache

Step 5: Deep Dive

Pick one:

- Caching
- DB scaling
- Rate limiting
- Messaging

Step 6: Failure Handling

- What if DB fails?
- What if cache is down?
- What if service crashes?

Step 7: Scaling

- Horizontal scaling
- Load balancing
- Sharding
- CDN

Step 8: Trade-offs & Cost

- Performance vs cost
- Consistency vs availability

4. Key Interview Skills

Zoom In / Zoom Out

Ability to:

- Start high-level
- Dive deep into components

Use Industry Language

Say things like:

- “stateless services”
- “eventual consistency”
- “cache invalidation”

Think Out Loud

Always explain reasoning:

- Why this design?
- Why not alternatives?

5. Common Mistakes

- Jumping into design without requirements
- Not discussing trade-offs
- Ignoring failures
- No scalability discussion
- No monitoring/logging
- Overcomplicating early

6. Golden Rules

- Start simple → scale later
- Always mention bottlenecks
- Always include caching & load balancing
- Always consider failures
- Always discuss trade-offs

7. Sample Opening

You can literally say:

“Before jumping into design, I’d like to clarify functional and non-functional requirements such as expected QPS, latency targets, and consistency requirements. Then I’ll propose a simple high-level design and gradually scale it.”

This creates a **very strong first impression**

8. Final Summary

- Clarify requirements first
- Everything is a trade-off
- Drive the conversation
- Discuss failures, monitoring, scaling
- Use diagrams
- Start simple → evolve